

Lab No: 12

* Lab title: **Pointers**

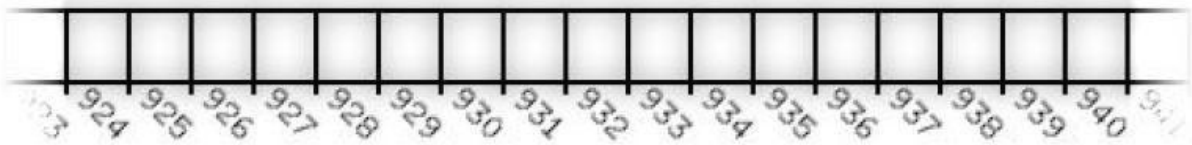
○ Learning objectives:

The following topics will be covered in this lab session

- Computer Memory and variable concept
- Pointer Introduction
- Pointer declaration
- Reference and Dereference Operators
- Pointer example and its explanation
- Pointer Arithmetic's
- Sending Pointers as Arguments to Functions

⇒ **Computer Memory:**

Essentially, the computer's memory is made up of bytes. Each byte has a number, an address, associated with it. The picture below represents several bytes of a computer's memory. In the picture, addresses 924 thru 940 are shown.



⇒ **Variable and Computer Memory:**

A variable in a program is something with a name, the value of which can vary. The way the compiler handles this is that it assigns a specific block of memory within the computer to hold the value of that variable. The size of that block depends on the range over which the variable is allowed to vary. For example, on 32 bit PC's the size of an integer variable is 4 bytes. On older 16 bit PCs integers were 2 bytes.

```
#include<iostream>
using namespace std;
int main()
{
    float fl=3.14;
    cout<<fl;
    cin>>fl;
    return 0;
}
```

The output for this program will be



At line (4) in the program above, the computer reserves memory for `fl`. Depending on the computer's architecture, a float may require 2, 4, 8 or some other number of bytes. In our example, we'll assume that a float requires 4 bytes.



- When `fl` is used in line (5), two distinct steps occur:
 1. The program finds and grabs the address reserved for `fl`--in this example 924.
 2. The contents stored at that address are retrieved

The illustration that shows 3.14 in the computer's memory can be misleading. Looking at the diagram, it appears that "3" is stored in memory location 924, "." is stored in memory location 925, "1" in 926, and "4" in 927. Keep in mind that the computer actually converts the floating-point number 3.14 into a set of ones and zeros. Each byte holds 8 ones or zeros. So, our 4-byte float is stored as 32 ones and zeros (8 per byte times 4 bytes). Regardless of whether the number is 3.14, or -273.15, the number is always stored in 4 bytes as a series of 32 ones and zeros.

➤ Pointer:

In C++ a pointer is a variable that points to or references a memory location in which data is stored. A pointer is a variable that **points** to another variable. This means that a pointer holds the memory address of another variable. Put another way, the pointer does not hold a value in the traditional sense; instead, it holds the address of another variable. A pointer "points to" that other variable by holding a copy of its address. Because a pointer holds an address rather than a value, it has two parts. The pointer itself holds the address and that address points to a value.

➤ Pointer declaration

A pointer is a variable that contains the memory location of another variable. The syntax is as shown below. You start by specifying the type of data stored in the location identified by the pointer. The asterisk tells the compiler that you are creating a pointer variable. Finally you give the name of the variable.

`Data_type *variable_name`

Such a variable is called a **pointer variable** (for reasons which hopefully will become clearer a little later).

In C++ when we define a pointer variable we do so by preceding its name with an asterisk. In C++ we also give our pointer a type which, in this case, refers to the type of data stored at the address we will be storing in our pointer. For example, consider the variable declaration:

```
int *ptr;
```

```
int k;
```

ptr is the name of our variable (just as **k** is the name of our integer variable). The '*' informs the compiler that we want a pointer variable, i.e. to set aside however many bytes is required to store an address in memory. The **int** says that we intend to use our pointer variable to store the address of an integer.

➤ Referencing Operator

Suppose now that we want to store in **ptr** the address of our integer variable **k**. To do this we use the unary **&** operator and write:

```
ptr = &k;
```

What the **&** operator does is retrieve the address of **k**, and copies that to the contents of our pointer **ptr**.

Now, **ptr** is said to "point to" **k**.

➤ Dereferencing operator:

The "dereferencing operator" is the asterisk and it is used as follows:

```
*ptr = 7;
```

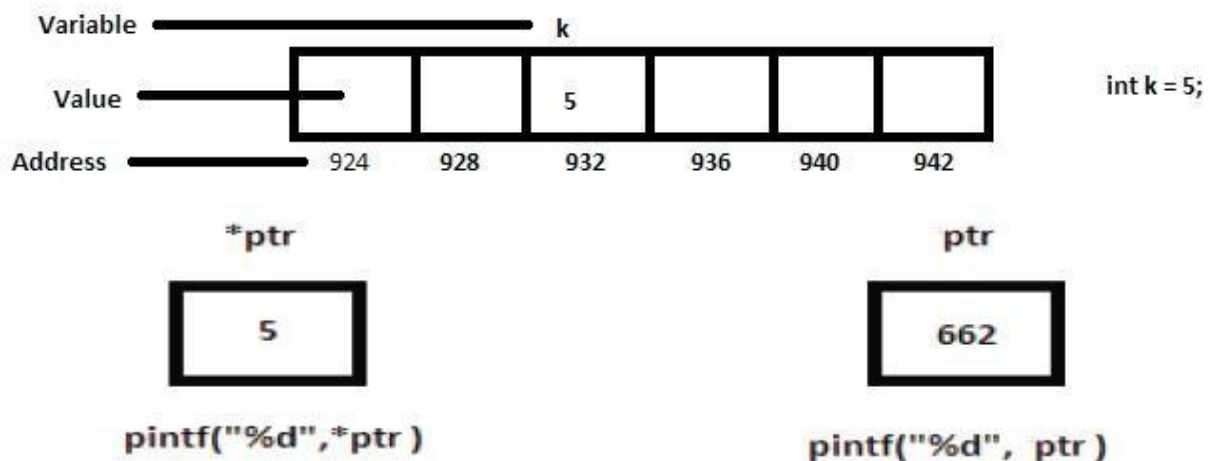
will copy 7 to the address pointed to by **ptr**. Thus if **ptr** "points to" (contains the address of) **k**, the above statement will set the value of **k** to 7. That is, when we use the '*' this way we are referring to the value of that which **ptr** is pointing to, not the value of the pointer itself.

Similarly, we could write:

```
cout<<*ptr<<endl;
```

to print to the screen the integer value stored at the address pointed to by **ptr**.

- Here is graphical representation of Pointers.



- **Output of this listing is as follows:**



```
C:\Users\PC\Downloads\Listing 1.exe
First Value 10
Second Value 20
```

Lab Tasks

- **Task 1: Pointer Declaration and Initialization**
 1. Declare an integer variable called "num" and initialize it with a value of your choice.
 2. Declare a pointer variable called "ptr" of type integer.
 3. Initialize the pointer variable to point to the address of the "num" variable.
 4. Display the value of "num" and the value pointed to by "ptr" using the de reference

↩ Input:

```
#include <iostream>
using namespace std;
int main() {
    int num = 42; // You can choose any value
    int *ptr;
    ptr = &num;
    cout << "The value of num: " << num << endl;
    cout << "The value pointed to by ptr: " << *ptr << endl;
    return 0;
}
```

↩ Output:

```
The value of num: 42
The value pointed to by ptr: 42
```

• Task 2: Pointer Arithmetic:

1. Declare an integer array called "numbers" with some values of your choice.
2. Declare a pointer variable called "ptr" of type integer and initialize it to point to the first element of the "numbers" array.
3. Use pointer arithmetic to access and display the elements of the array using the pointer.

↩ Input:

```
#include <iostream>
using namespace std;
int main() {
    int numbers[] = {10, 20, 30, 40, 50};
    int *ptr = numbers;
    cout << "Array elements using pointer arithmetic:" << endl;
    for (int i = 0; i < 5; ++i) {
        cout << *(ptr + i) << endl;
    }
    return 0;
}
```

➡ **Output:**

```
Array elements using pointer arithmetic:
10
20
30
40
50
```

